UNIVERSITY OF CALIFORNIA, BERKELEY
DEPARTMENT OF STATISTICS

Homework #8

Student name: *Shizhe Zhang*

Course: *EECS 227AT* – Professor: *Gireeja Ranade*
Date: *March 17, 2026*

Problem 1. Convergence of Gradient Descent for Ridge Regression

Let $A \in \mathbb{R}^{m \times n}$, $\vec{y} \in \mathbb{R}^m$, and $\lambda > 0$. Consider a slight variation of the ridge regression problem where the least squares loss is normalized by the number of data points:

$$\min_{\vec{x} \in \mathbb{R}^n} f_\lambda(\vec{x})$$

where

$$f_\lambda(\vec{x}) \triangleq \frac{1}{2}\left\{\frac{1}{m}\|A\vec{x} - \vec{y}\|_2^2 + \lambda\|\vec{x}\|_2^2\right\}. \tag{1}$$

In this problem, we will examine the behavior of gradient descent (GD) on this problem, and in particular the interplay between the learning rate $\eta > 0$ and regularization parameter $\lambda > 0$ in determining convergence.

(a) Show that the unique solution to the problem in Equation (1) is

$$\vec{x}_\lambda^\star = (A^\top A + \lambda m I)^{-1} A^\top \vec{y}. \tag{2}$$

(b) Show that the GD update

$$\vec{x}_{t+1} = \vec{x}_t - \eta\left(\frac{1}{m}A^\top(A\vec{x}_t - \vec{y}) + \lambda\vec{x}_t\right) \tag{3}$$

can be rearranged into the form

$$\vec{x}_{t+1} - \vec{x}_\lambda^\star = \left(I - \eta\left(\frac{A^\top A}{m} + \lambda I\right)\right)(\vec{x}_t - \vec{x}_\lambda^\star). \tag{4}$$

Use this to show that

$$\vec{x}_t - \vec{x}_\lambda^\star = \left(I - \eta\left(\frac{A^\top A}{m} + \lambda I\right)\right)^t (\vec{x}_0 - \vec{x}_\lambda^\star) \tag{5}$$

for every positive integer t.

(c) We now discuss the insight that the SVD can give us regarding the convergence of GD. Let

$$A = U\Sigma V^\top$$

be a full SVD of A. Let

$$\vec{z}_t = V^\top \vec{x}_t$$

and

$$\vec{z}_\lambda^\star = V^\top \vec{x}_\lambda^\star.$$

Show that

$$\vec{z}_t - \vec{z}_\lambda^\star = \left(I - \eta \left(\frac{\Sigma^\top \Sigma}{m} + \lambda I \right) \right)^t (\vec{z}_0 - \vec{z}_\lambda^\star). \tag{6}$$

Moreover, show that for each $i \in \{1, \ldots, n\}$,

$$(z_t)_i - (z_\lambda^\star)_i = \left(1 - \eta \left(\frac{\sigma_i\{A\}^2}{m} + \lambda \right) \right)^t ((z_0)_i - (z_\lambda^\star)_i) \tag{7}$$

where $\sigma_i\{A\}$ is the ith largest singular value of A.

This shows that the rate of convergence of $\vec{z}_t$ to $\vec{z}_\lambda^\star$ along the ith component is influenced by the interaction between $\sigma_i\{A\}$, λ, and η, but critically no other singular values. Thus, one considers the V basis to be the natural basis for thinking about GD for ridge regression.

(d) Show that

$$\lim_{t \to \infty} \vec{z}_t = \vec{z}_\lambda^\star$$

for all initializations $\vec{x}_0 = V\vec{z}_0$ if and only if

$$\max_{i \in \{1, \ldots, n\}} \left| 1 - \eta \left(\frac{\sigma_i\{A\}^2}{m} + \lambda \right) \right| < 1. \tag{8}$$

Use this to show that GD converges for all initializations $\vec{x}_0$ if and only if

$$\eta \in \left(0, \frac{2m}{\sigma_{\max}\{A\}^2 + \lambda m} \right) \tag{9}$$

where $\sigma_{\max}\{A\} = \sigma_1\{A\}$ is the largest singular value of A.

(e) (OPTIONAL) One way we can derive an optimal learning rate $\eta^\star$ is to minimize the largest rate of convergence:

$$\eta^\star \in \arg\min_{\eta \in \mathbb{R}} \max_{i \in \{1, \ldots, n\}} \left| 1 - \eta \left(\frac{\sigma_i\{A\}^2}{m} + \lambda \right) \right|. \tag{10}$$

One important property of $\eta^\star$ is that it makes the rates of convergence

$$\left| 1 - \eta \left(\frac{\sigma_i\{A\}^2}{m} + \lambda \right) \right|$$

associated with the largest and smallest singular values of A equal.

Use this property to show that

$$\eta^\star = \frac{2m}{\sigma_{\max}\{A\}^2 + \sigma_{\min}\{A\}^2 + 2\lambda m}. \tag{11}$$

NOTE: There are several useful notions of optimal learning rate; this is just one of them.

(f) The attached notebook, `gd_convergence.ipynb`, will examine the computational aspects of GD on ridge regression. Implement the GD and stochastic gradient descent (SGD) functions at the top of the notebook, which are marked with TODOs.

(g) Click through the notebook and run the sections $n = 1$, $n = 2$, and $n \gg 2$. Change the values of λ and η and re-run the cells a few times. Write down your observations about how the convergence of GD works under different values of λ and η.

(h) In the sections $n = 1$, $n = 2$, and $n \gg 2$, change the calls to GD to instead call SGD. Write down your observations about how the convergence of SGD works under different values of λ and η. Compare the behavior of GD and SGD.

(i) You might have noticed that if we think of convergence in the last iterate sense, i.e.,

$$\lim_{T\to\infty} \vec{x}_T = \vec{x}^\star_\lambda,$$

then SGD rarely converges.

This is because even if we reach the global optimum, the gradient estimate used by SGD is in general nonzero, and so the iterates end up bouncing around near the optimum.

Another different, weaker notion of convergence under which one might show that SGD actually does converge is convergence in time average, i.e., $\lim_{T\to\infty} \bar{x}_T = \vec{x}^\star_\lambda$ where $\bar{x}_T \triangleq \frac{1}{T}\sum_{t=1}^{T} x_t$.

Visualize this by adding the argument `time_avg=True` to each plotting function; the plot will now visualize the sequence of $\bar{x}_t$. Re-run the notebook. Write down your observations, especially regarding the stability of SGD and convergence in the last-iterate sense versus the time-average sense.

Answer. (a)

We consider

$$f_\lambda(\vec{x}) = \frac{1}{2}\left(\frac{1}{m}\|A\vec{x} - \vec{y}\|_2^2 + \lambda\|\vec{x}\|_2^2\right).$$

First compute the gradient.

$$\nabla_x \|A\vec{x} - \vec{y}\|_2^2 = 2A^\top(A\vec{x} - \vec{y})$$

Thus

$$\nabla f_\lambda(\vec{x}) = \frac{1}{m}A^\top(A\vec{x} - \vec{y}) + \lambda\vec{x}.$$

Setting the gradient to zero,

$$\frac{1}{m}A^\top(A\vec{x} - \vec{y}) + \lambda\vec{x} = 0.$$

Multiplying both sides by m,

$$A^\top A\vec{x} - A^\top\vec{y} + \lambda m\vec{x} = 0.$$

Rearranging,

$$(A^\top A + \lambda m I)\vec{x} = A^\top \vec{y}.$$

Since $A^\top A + \lambda m I$ is positive definite for $\lambda > 0$, it is invertible, giving

$$\vec{x}_\lambda^\star = (A^\top A + \lambda m I)^{-1} A^\top \vec{y}.$$

(b)
The GD update is

$$\vec{x}_{t+1} = \vec{x}_t - \eta \left(\frac{1}{m} A^\top (A\vec{x}_t - \vec{y}) + \lambda \vec{x}_t \right).$$

Rewrite the update:

$$\vec{x}_{t+1} = \vec{x}_t - \eta \left(\frac{A^\top A}{m} \vec{x}_t - \frac{A^\top \vec{y}}{m} + \lambda \vec{x}_t \right).$$

Using the optimality condition

$$(A^\top A + \lambda m I)\vec{x}_\lambda^\star = A^\top \vec{y},$$

we obtain

$$\frac{A^\top \vec{y}}{m} = \left(\frac{A^\top A}{m} + \lambda I \right) \vec{x}_\lambda^\star.$$

Substituting,

$$\vec{x}_{t+1} = \vec{x}_t - \eta \left(\frac{A^\top A}{m} + \lambda I \right) (\vec{x}_t - \vec{x}_\lambda^\star).$$

Thus

$$\vec{x}_{t+1} - \vec{x}_\lambda^\star = \left(I - \eta \left(\frac{A^\top A}{m} + \lambda I \right) \right) (\vec{x}_t - \vec{x}_\lambda^\star).$$

Applying the recursion repeatedly gives

$$\vec{x}_t - \vec{x}_\lambda^\star = \left(I - \eta \left(\frac{A^\top A}{m} + \lambda I \right) \right)^t (\vec{x}_0 - \vec{x}_\lambda^\star).$$

(c)
Then

$$A^\top A = V \Sigma^\top \Sigma V^\top.$$

Define

$$\vec{z}_t = V^\top \vec{x}_t, \qquad \vec{z}_\lambda^\star = V^\top \vec{x}_\lambda^\star.$$

Multiplying equation (5) by $V^\top$,

$$\vec{z}_t - \vec{z}_\lambda^\star = V^\top \left(I - \eta \left(\frac{A^\top A}{m} + \lambda I \right) \right)^t V(\vec{z}_0 - \vec{z}_\lambda^\star).$$

Since

$$V^\top A^\top A V = \Sigma^\top \Sigma,$$

we obtain

$$\vec{z}_t - \vec{z}_\lambda^\star = \left(I - \eta \left(\frac{\Sigma^\top \Sigma}{m} + \lambda I \right) \right)^t (\vec{z}_0 - \vec{z}_\lambda^\star).$$

Because $\Sigma^\top \Sigma$ is diagonal with entries $\sigma_i(A)^2$,

$$(z_t)_i - (z_\lambda^\star)_i = \left(1 - \eta \left(\frac{\sigma_i(A)^2}{m} + \lambda \right) \right)^t ((z_0)_i - (z_\lambda^\star)_i).$$

Thus each coordinate converges independently.

(d)

From part (c),

$$(z_t)_i - (z_\lambda^\star)_i = \left(1 - \eta \left(\frac{\sigma_i(A)^2}{m} + \lambda \right) \right)^t ((z_0)_i - (z_\lambda^\star)_i).$$

For convergence as $t \to \infty$, we require

$$\left| 1 - \eta \left(\frac{\sigma_i(A)^2}{m} + \lambda \right) \right| < 1$$

for all i.

This gives

$$-1 < 1 - \eta \left(\frac{\sigma_i(A)^2}{m} + \lambda \right) < 1.$$

The right inequality gives

$$\eta > 0.$$

The left inequality gives

$$\eta < \frac{2}{\frac{\sigma_i(A)^2}{m} + \lambda}.$$

For all i, the most restrictive condition occurs at $\sigma_{\max}(A)$. Thus

$$\eta < \frac{2}{\frac{\sigma_{\max}(A)^2}{m} + \lambda}.$$

Therefore GD converges for all initializations if

$$\eta \in \left(0, \frac{2m}{\sigma_{\max}(A)^2 + \lambda m} \right).$$

(e)
The optimal learning rate equalizes the largest and smallest convergence factors:

$$\left|1-\eta\left(\frac{\sigma_{\max}(A)^2}{m}+\lambda\right)\right| = \left|1-\eta\left(\frac{\sigma_{\min}(A)^2}{m}+\lambda\right)\right|.$$

This yields

$$1-\eta\left(\frac{\sigma_{\max}^2}{m}+\lambda\right) = -\left(1-\eta\left(\frac{\sigma_{\min}^2}{m}+\lambda\right)\right).$$

Simplifying,

$$2=\eta\left(\frac{\sigma_{\max}^2+\sigma_{\min}^2}{m}+2\lambda\right).$$

Thus

$$\eta^\star = \frac{2}{\frac{\sigma_{\max}^2+\sigma_{\min}^2}{m}+2\lambda} = \frac{2m}{\sigma_{\max}^2+\sigma_{\min}^2+2\lambda m}.$$

(g)
I re-ran the notebook for $n = 1, 2, 10$ while varying both λ and η. The empirical behavior matches the theory from part (d):

- For GD, increasing η generally speeds up convergence until a stability boundary is crossed.
- For moderate dimensions ($n = 1, 2$), GD converged reliably for a wide range of step sizes around $\eta^\star$.
- For higher dimension ($n = 10$), the instability is more visible: with small regularization ($\lambda = 0.01$), using a large step (e.g., $\eta \approx 1.5\eta^\star$) caused divergence (overflow/inf in the iterate error).
- Increasing λ improves conditioning and makes GD behavior more stable across step sizes.

In the convergence plots, GD trajectories are smooth and nearly geometric on a log scale, especially in the V basis, consistent with coordinate-wise contraction factors.

(h)
After replacing GD calls with SGD and sweeping λ, η again, the behavior changes qualitatively:

- SGD last iterates are noisy and oscillatory; they do not settle to a single point even when objective values are small.
- Larger η increases variance of the iterates and can lead to severe instability/divergence in high dimension (for $n = 10$ and small λ, large-step runs produced NaNs).
- Compared with GD, SGD is less stable at the same nominal scale of η, and convergence diagnostics based on $\|x_t - x_\lambda^\star\|$ are much worse for last iterates.

- Stronger regularization (λ larger) damps noise somewhat, but does not remove last-iterate oscillation.

So GD gives fast and clean deterministic convergence, while SGD trades that for noisy but inexpensive updates.

(i)

Using the plotting functions with `time_avg=True` confirms the expected phenomenon:

- Last-iterate SGD remains jittery around the optimum in both standard and V bases.
- Time-averaged SGD, $\bar{x}_t$, is much more stable and trends downward in error/objective gap, often by one or more orders of magnitude relative to last-iterate error.
- This effect is strongest in noisy/high-dimensional settings ($n = 10$), where averaging converts highly variable trajectories into steadily improving ones.

Therefore, for SGD in this problem, "convergence in last iterate" is typically poor, while "convergence in time average" is the practically meaningful notion.

2. Convexity of Rank 1 Matrices

First, consider the set of all 2×2 matrices with diagonal elements $(1, 2)$:

$$S = \left\{ \begin{bmatrix} 1 & x \\ y & 2 \end{bmatrix} \mid x, y \in \mathbb{R} \right\}. \tag{12}$$

(a) Is set S convex? If so, provide a proof, and if not, provide a counterexample.

(b) Suppose we now wish to define $S_1 \subset S$, the set of all rank-1 matrices in S. Write out conditions on x and y to define S_1 explicitly.

(c) Is set S_1 convex? If so, provide a proof, and if not, provide a counterexample.

(d) Consider the optimization problem

$$\min_{A \in S_1} \|A\|_F^2 \tag{13}$$

which is equivalent to

$$\min_{A \in S} \|A\|_F^2 \tag{14}$$

subject to

$$\operatorname{rank}(A) = 1. \tag{15}$$

Is this optimization problem convex?

Answer. First consider the set

$$S = \left\{ \begin{bmatrix} 1 & x \\ y & 2 \end{bmatrix} \middle| \, x, y \in \mathbb{R} \right\}.$$

(a)

Yes, S is convex.

Let

$$A = \begin{bmatrix} 1 & x_1 \\ y_1 & 2 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & x_2 \\ y_2 & 2 \end{bmatrix}$$

be two matrices in S. Let $\theta \in [0, 1]$.

Consider the convex combination

$$\theta A + (1 - \theta) B.$$

We compute

$$\theta A + (1 - \theta) B = \begin{bmatrix} \theta + (1 - \theta) & \theta x_1 + (1 - \theta) x_2 \\ \theta y_1 + (1 - \theta) y_2 & 2\theta + 2(1 - \theta) \end{bmatrix}.$$

Simplifying,

$$= \begin{bmatrix} 1 & \theta x_1 + (1 - \theta) x_2 \\ \theta y_1 + (1 - \theta) y_2 & 2 \end{bmatrix}.$$

Since $\theta x_1 + (1 - \theta) x_2$ and $\theta y_1 + (1 - \theta) y_2$ are real numbers, the resulting matrix is still in S. Therefore S is convex.

(b)
Let

$$A = \begin{bmatrix} 1 & x \\ y & 2 \end{bmatrix}.$$

For a 2×2 matrix to have rank 1, its determinant must be zero. Thus

$$\det(A) = 0.$$

Compute the determinant:

$$\det(A) = (1)(2) - xy = 2 - xy.$$

Setting this equal to zero,

$$xy = 2.$$

$$S_1 = \left\{ \begin{bmatrix} 1 & x \\ y & 2 \end{bmatrix} \;\middle|\; xy = 2 \right\}.$$

(c)
The set S_1 is not convex.
Consider the matrices

$$A_1 = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 1 & 2 \\ 1 & 2 \end{bmatrix}.$$

Both satisfy $xy = 2$, so they belong to S_1.
Now consider their average:

$$A = \frac{1}{2}A_1 + \frac{1}{2}A_2 = \begin{bmatrix} 1 & 1.5 \\ 1.5 & 2 \end{bmatrix}.$$

For this matrix

$$xy = (1.5)(1.5) = 2.25 \neq 2.$$

Thus $A \notin S_1$, meaning the convex combination of two matrices in S_1 is not necessarily in S_1. Therefore S_1 is not convex.

(d)
The optimization problem is

$$\min_{A \in S_1} \|A\|_F^2.$$

$$\|A\|_F^2 = \sum_{i,j} A_{ij}^2,$$

which is a convex function.
However, the feasible set S_1 is not convex (from part (c)) because of the rank constraint.
Thus, this problem is not convex.

3. Positive Semi-Definite Cone

Let $A, B \in \mathbb{R}^{n\times n}$ be symmetric positive semidefinite matrices.

Show that

$$\langle A, B\rangle \geq 0$$

where

$$\langle A, B\rangle = \mathrm{tr}(AB^\top)$$

is the usual inner product on matrices.

HINT: Try an eigendecomposition of $B = \sum_{i=1}^n \lambda_i u_i u_i^\top$ and use the trace formula of the inner product.

Answer. Since B is symmetric, we have $B^\top = B$, so

$$\langle A, B\rangle = \mathrm{tr}(AB).$$

Because B is positive semidefinite, it admits the eigendecomposition

$$B = \sum_{i=1}^n \lambda_i u_i u_i^\top$$

where $\lambda_i \geq 0$ are the eigenvalues and $\{u_i\}$ are orthonormal eigenvectors.

Substituting this into the inner product,

$$\langle A, B\rangle = \mathrm{tr}\left(A \sum_{i=1}^n \lambda_i u_i u_i^\top\right).$$

Using linearity of the trace,

$$= \sum_{i=1}^n \lambda_i \mathrm{tr}(A u_i u_i^\top).$$

Using the cyclic property of trace,

$$\mathrm{tr}(A u_i u_i^\top) = \mathrm{tr}(u_i^\top A u_i) = u_i^\top A u_i.$$

Thus

$$\langle A, B\rangle = \sum_{i=1}^n \lambda_i\, u_i^\top A u_i.$$

Since A is positive semidefinite,

$$u_i^\top A u_i \geq 0$$

for all i, and since $\lambda_i \geq 0$, every term in the sum is nonnegative. Therefore

$$\langle A, B\rangle \geq 0.$$

Hence, the inner product of two symmetric positive semidefinite matrices is always nonnegative.

4. Visualizing the Dual Problem

Use the associated Google Colab notebook; complete the code where designated and answer the questions given in the space provided.

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
%matplotlib inline
```

Duality

In this problem we look at the primal and dual problems for simple cases and get insights into duality.

Helper functions

In [3]:
```
## DO NOT MODIFY THIS CODE
# Wrapper function to get primal functions
def get_primal(subpart):
    if subpart == 0:
        x = np.linspace(-2, 8, 100)
        return x, get_primal_lp(x), get_p_opt_lp(x), get_primal_constraint_lp()
    elif subpart == 1:
        x, y = np.linspace(-2, 2, 100), np.linspace(-2, 2, 100)
        X, Y = np.meshgrid(x, y)
        return X, Y, get_primal_e(X, Y), get_primal_constraint_e(x, y), get_p_opt_e(np.linspace(0, 10, 100))
    elif subpart == 2:
        x = np.linspace(-5, 5, 100)
        return
    else:
        print("Subpart must be 0, 1, or 2.")

# Wrapper function to get dual functions
def get_dual(subpart):
    if subpart == 0:
        lam = np.linspace(-2, 8, 100)
        dual_constraints = get_dual_constraints_lp()
        return lam, get_dual_lp(lam), dual_constraints
    elif subpart == 1:
        lam = np.linspace(10**-10, 4, 100)
```

```
        return lam, get_dual_e(lam)
    elif subpart == 2:
        return
    else:
        print("Subpart must be 0, 1, or 2.")

# Helper function for plotting. Do not change this function.
def visualize(subpart):
    p = get_primal(subpart)
    d = get_dual(subpart)
    if subpart == 0: # 1d primal and dual variables
        (lam, dual, dual_constraints) = d
        x, primal, p_opt, primal_constraint = p
        fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, sharey=True, figsize = [12,6])
        ax1.plot(x, primal, color="b", label=r'$f(x)$')
        ax1.plot(x, p_opt, color="k", linestyle="dotted", label=r'$p^* = 6$')
        ax1.axvspan(-2, primal_constraint, alpha=0.5, facecolor="r")
        ax1.set_xlim([-2, 8])
        ax1.legend(loc="upper left")
        ax1.set_title("Primal")
        ax1.set_xlabel(r'$x$')
        ax1.set_ylabel(r'$f(x)$')


        ax2.plot(lam, dual, color="g", label=r'$g(\lambda)$')
        ax2.plot(x, p_opt, color="k", linestyle="dotted", label=r'$p^* = 6$')
#         ax2.plot(dual_constraints[0], dual_constraints[1], color="y")
        ax2.axvspan(-2, dual_constraints[0], alpha=0.5, facecolor="r")
        ax2.axvspan(dual_constraints[1], 8, alpha=0.5, facecolor="r")

        ax2.legend(loc="upper left")
        ax2.set_title("Dual")
        ax2.set_xlabel(r'$\lambda$')
        ax2.set_ylabel(r'$g(\lambda)$')

    elif subpart == 1: # 2d primal variables
        (lam, dual) = d
        X, Y, primal, primal_constraint, p_opt = p
        plt3d = plt.gcf().add_subplot(projection='3d')
        plt3d.plot_surface(X, Y, primal_constraint, alpha=.2, color = 'red')
        ax = plt.gca()
```

```
        ax.contour3D(X, Y, primal, 50, cmap='binary')
        ax = plt.gca()
        ax.set_xlabel(r'$x$')
        ax.set_ylabel(r'$y$')
        ax.set_zlabel(r'$f(x,y)$')
        ax.set_title("Primal")

        fig2, (ax1, ax2, ax3) = plt.subplots(nrows=1, ncols=3, sharey=True, figsize = [12,6])

        x = np.linspace(0, 2, 100)
        ax1.set_title(r'Cross section: $f(x,y)$ s.t. $x + y = 1$')
        ax1.plot(x, np.exp(x) + np.exp(1-x), label=r'$f(x,y)$')
        ax1.plot(x, p_opt, color="k", linestyle="dotted", label=r'$p^*$')
        ax1.set_xlabel(r'$x$')
        ax1.legend(loc="upper right")

        ax2.set_title(r'Cross section: $f(x,y)$ s.t. $y = \frac{1}{2}$')
        ax2.plot(x, np.exp(x) + np.exp(.5), label=r'$f(x,y)$')
        ax2.plot(x, p_opt, color="k", linestyle="dotted", label=r'$p^*$')
        ax2.axvspan(0, .5, alpha=0.5, facecolor="r")
        ax2.set_xlabel(r'$x$')
        ax2.legend(loc="upper right")

        ax3.plot(lam, dual, label=r'$g(\lambda)$')
        ax3.plot(lam, p_opt, color="k", linestyle="dotted", label=r'$p^*$')
        ax3.set_title("Dual")
        ax3.set_xlabel(r'$\lambda$')
        ax3.legend(loc="upper right")

    elif subpart == 2:
        return
    else:
        print("Subpart must be 0, 1, or 2.")
    plt.legend()
```

Problem (1): A Linear program

Problem formulation

Consider the problem of minimizing a linear objective subject to a linear constraint. Suppose a is a positive scalar.

$$p^* = \min_{x \geq 0} ax$$
$$\text{s.t. } x \geq b.$$

We consider the case where $a = 2$ and $b = 3$. In this case clearly the optimal primal solution is $x^* = 3$ and $p^* = 6$.

Constructing the primal

In [4]:
```
## DO NOT MODIFY THIS CODE
a = 2
b = 3

##Get primal for plottting
def get_primal_lp(x):
    """ x: array of scalars """
    return np.multiply(x, a)

def get_p_opt_lp(x):
    """
    Used for plotting the optimal value p*.
    Input:
    Output:
    """
    return [a*b for _ in x]

def get_primal_constraint_lp():
    """
    Used for plotting feasible x region
    """
    return b
```

We can formulate the dual problem for the LP which involves maximizing $g(\lambda_1, \lambda_2)$ where λ_1 is the dual variable corresponding to the constraint $x \geq 3$ and λ_2 is the dual variable corresponding to the constraint $x \geq 0$. Then, we solve it to obtain d^*.

The Lagrangian is given by

$$\begin{aligned}\mathcal{L}(x, \lambda_1, \lambda_2) &= 2x + \lambda_1(-x+3) + \lambda_2(-x) \\ &= x(2 - \lambda_1 - \lambda_2) + 3\lambda_1.\end{aligned}$$

Then for $\lambda_1 \geq 0$, $\lambda_2 \geq 0$,

$$g(\lambda_1, \lambda_2) = \inf_x \mathcal{L}(x, \lambda_1, \lambda_2).$$

Observe that if $\lambda_1 + \lambda_2 \neq 2$ then $g(\lambda_1, \lambda_2)$ will be $-\infty$, attained by taking x to ∞ or $-\infty$.

Therefore, the dual problem is

$$\begin{aligned}d^* = &\max_{\lambda_1 \geq 0,\ \lambda_2 \geq 0} 3\lambda_1 \\ &\text{s.t. } \lambda_1 + \lambda_2 = 2.\end{aligned}$$

To maximize $3\lambda_1$ we should set $\lambda_1 = 2$ and $\lambda_2 = 0$. This gives $d^* = 6$.

Next we will consider the Lagrangian only with respect to the constraint $x \geq 3$ and form the dual problem.

We formulate the dual problem for the LP which involves maximizing $g(\lambda)$ where λ is the dual variable corresponding to the constraint $x \geq 3$. Then, we compare the dual problem to that of the previous one.

Similar to the previous part, the Lagrangian is

$$\begin{aligned}\mathcal{L}(x, \lambda) &= 2x + \lambda(-x+3) \\ &= x(2-\lambda) + 3\lambda.\end{aligned}$$

The dual problem is given by

$$d^* = \max_{\lambda \geq 0} \min_{x \geq 0} x(2-\lambda) + 3\lambda.$$

Note that if $2 - \lambda < 0, g(\lambda) = -\infty$, attained by taking x to ∞. Therefore, the optimal λ^* must satisfy $\lambda \leq 2$, and x must be 0.

That is, the dual problem simplifies to

$$d^* = \max_{\lambda \geq 0} 3\lambda$$
$$\text{s.t. } \lambda \leq 2.$$

Note that while this dual problem appears different from that of the previous part, we can eliminate λ_2 using the equality and the inequality constraint on λ_2. Thus the two problems are essentially the same.

For problem without strong duality: Note that if strong duality does not hold it need not always be true, that the dual problem obtained by dualizing different constraints are equivalent. For example consider the following optimization problem,

$$p^* = \min_{x \geq 0} x^3$$
$$\text{s.t. } x \geq 1.$$

The problem is not convex since the objective function, x^3, is not convex. As an ungraded (and fun!) exercise that is not part of the homework, formulate the dual problem and compare the dual optimal values for the following 2 cases:

1. Form the dual only with respect to constraint $x \geq 1$
2. Form the dual with respect to both constraints, $x \geq 0$ and $x \geq 1$.

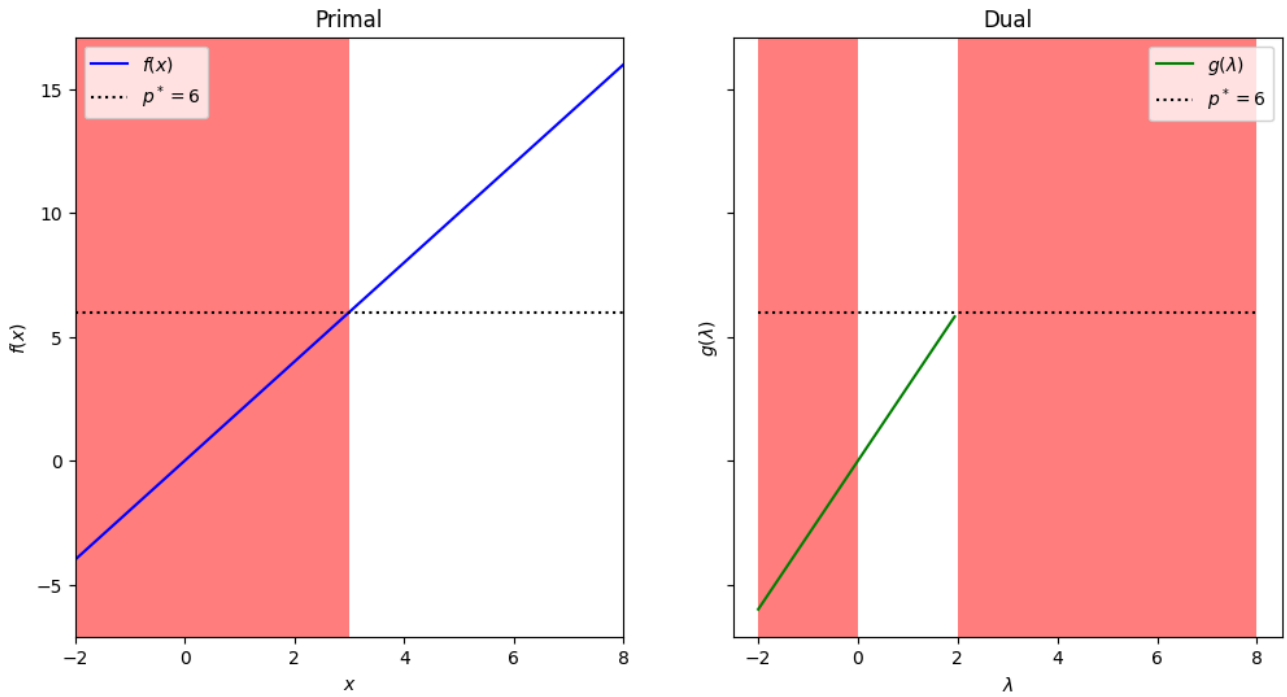
Fill in the get_dual_lp function and get_dual_constraints function based on the dual problem:

$$d^* = \max_{\lambda \geq 0} 3\lambda$$
$$\text{s.t. } \lambda \leq 2.$$

In [11]:
```
### Constructing the dual

def get_dual_lp(lam):
    """
    The objective function of the dual problem. Takes in lam and evaluate g(lam).
    Input: lam: array of scalars representing dual variable
    Output: g(lam): function value can be computed using variables of the linear program
    """
```

```
    ##############################################################################
    ### TODO: Replace the return value with the dual function, g(lambda)
    return np.where(lam <= a, b * lam, -np.inf)
    ### end TODO
    ##############################################################################



def get_dual_constraints_lp():
    """
    Get the bounds of the constraints that limit the feasible set of the dual variable.
    Output: Return two floats, lam_low, lam_high so that the dual variable is
    constrained as lam_low <= lam <= lam_high
    """

    ##############################################################################
    ### TODO: Set the bounds for the dual variable
    lam_low = 0
    lam_high = a

    ### end TODO
    ##############################################################################

    return lam_low, lam_high
```

In [12]:
```
visualize(0)
```



TODO:

Based on the visualization of the primal and dual problem, where red region denotes region of infeasibility, what is the dual optimal solution λ^* ? Does strong duality hold?

The optimal lambda = 2. Strong duality hold.

Problem (2): Minimizing exponentials in $\mathbb{R}^2$

Problem formulation

Consider the following problem, where $z = [x, y]$ is the two-dimensional primal variable.

$$p^* = \min_{x,y} e^x + e^y$$
$$\text{s.t. } x + y \geq 1.$$

We first solve the primal problem to get p^* and primal optimal solutions x^*, y^*.

Since objective function is increasing in both x and y the optimal solution must satisfy $x + y = 1$. If (x, y) satisifies strict inequality with respect to constraint then we can decrease either x or y while not violating constraint and simultaneously decrease objective value.

Substituting for $y = 1 - x$ and differentiating with respect to x we obtain $x^* = 0.5$ which gives us $y^* = 0.5$ and $p^* = 2e^{0.5} \approx 3.3$.

Constructing the primal

In [13]:
```
def get_primal_e(x, y):
    """ x, y: arrays of scalars """
    return np.add(np.exp(x), np.exp(y))

def get_primal_constraint_e(x, y):
    return np.reshape(x + y - 1, [-1,1])

def get_p_opt_e(x):
    """ Used for plotting the optimal value p*. """
    return [2 * np.exp(.5) for _ in x]
```

TODO:

Does Slater's condition hold? Is the problem convex? Does strong duality hold?

The objective $f(x, y) = e^x + e^y$ is a sum of convex functions, hence convex. The constraint $x + y \geq 1$ is a linear (affine) constraint, which defines a convex feasible set. Therefore the problem is convex.

For Slater's condition to hold, we need a strictly feasible point, i.e., a point (x, y) such that $x + y > 1$ strictly. For example, $(x, y) = (1, 1)$ satisfies $1 + 1 = 2 > 1$. Therefore Slater's condition holds.

Since the problem is convex and Slater's condition holds, by Slater's theorem strong duality holds.

Next, we formulate the dual problem for the minimizing exponentials problem, which involves maximizing $g(\lambda)$ where λ is the dual variable corresponding to the constraint $x + y \geq 1$.

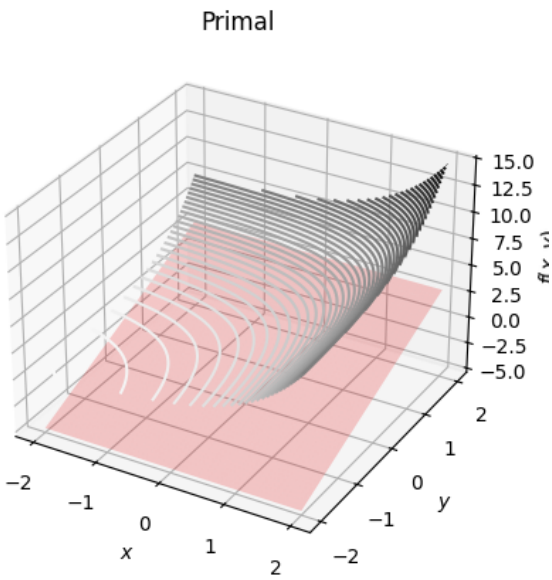
After deriving $g(\lambda)$, we find that the dual problem is given by

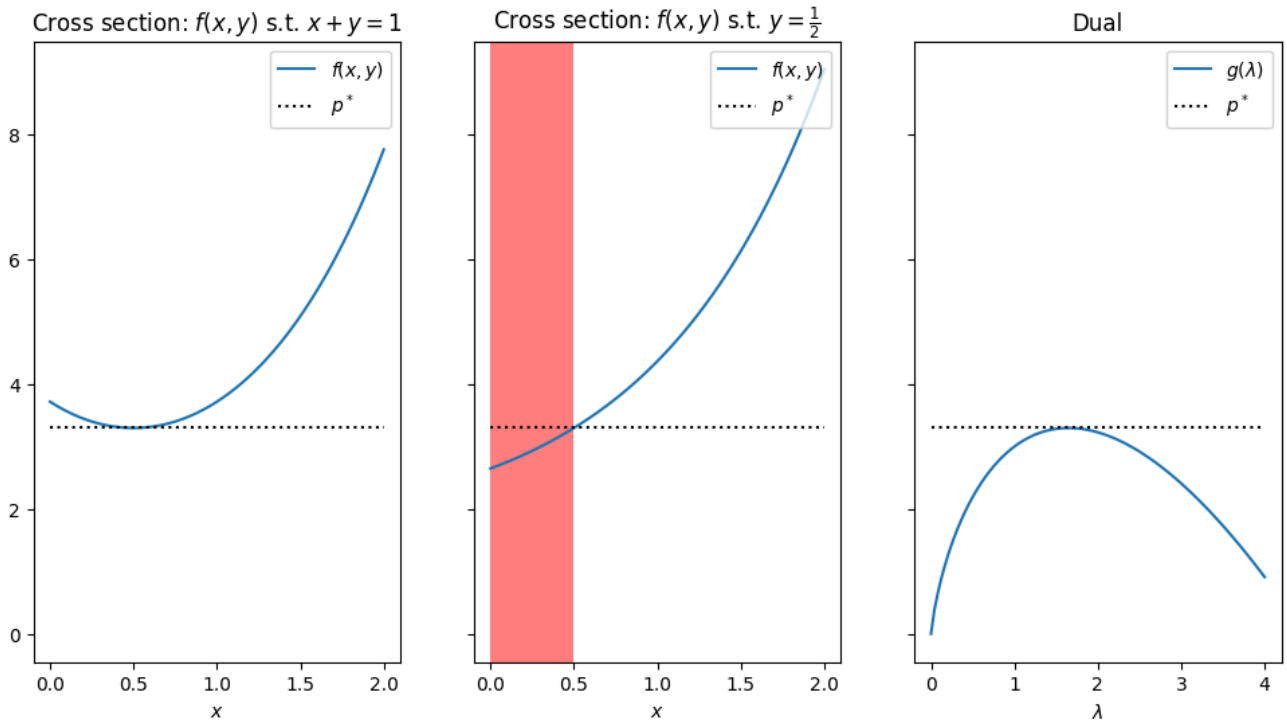
$$\begin{aligned} d^* = & \max_{\lambda} \lambda(3 - 2\ln\lambda) \\ & \text{s.t. } \lambda \geq 0. \end{aligned}$$

Fill in the get_dual_lp function function based on the given dual problem

In [14]:
```
def get_dual_e(lam):
    """
    The objective function of the dual problem.
    lam: array of scalars
    """

    ##############################################################################
    ### TODO: Replace the return value with the dual function, g(lambda)
    return lam * (3 - 2 * np.log(lam))
    ### end TODO
    ##############################################################################



def get_dual_constraints_e():
    return (0,5)
```

In [15]:
```
visualize(1)
```


Primal
15.0
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−2
−1
0
1
2
x
−2
−1
0
1
2
y



TODO:

Is the dual optimal value d^* equal to $p*$?

Yes, this is because strong duality holds.

5. Dual of an LP

Consider the general form of a linear program:

$$\min_{\vec{x}} c^\top \vec{x} \tag{16}$$

subject to

$$A\vec{x} = \vec{b}. \tag{17}$$

(a) Write the Lagrangian function $L(\vec{x}, \vec{\mu})$.
(b) Write the Lagrangian dual function $g(\vec{\mu})$.
(c) Write the dual problem.

Answer. (a)

$$L(\vec{x}, \vec{\mu}) = c^\top \vec{x} + \vec{\mu}^\top (A\vec{x} - \vec{b}).$$

We can rearrange it as

$$L(\vec{x}, \vec{\mu}) = (c^\top + \vec{\mu}^\top A)\vec{x} - \vec{\mu}^\top \vec{b}.$$

(b)

$$g(\vec{\mu}) = \inf_{\vec{x}} L(\vec{x}, \vec{\mu}).$$

$$g(\vec{\mu}) = \inf_{\vec{x}} \left[(c^\top + \vec{\mu}^\top A)\vec{x} - \vec{\mu}^\top \vec{b} \right].$$

If $c^\top + \vec{\mu}^\top A \neq 0$, then the expression can be made arbitrarily negative by choosing $\vec{x}$ appropriately, so $g(\vec{\mu}) = -\infty$.

To obtain a finite value, we must have

$$c^\top + \vec{\mu}^\top A = A^\top \vec{\mu} + c = 0.$$

$$g(\vec{\mu}) = \begin{cases} -\vec{\mu}^\top \vec{b}, & A^\top \vec{\mu} + c = 0 \\ -\infty, & \text{otherwise.} \end{cases}$$

(c)

$$\max_{\vec{\mu}} -\vec{\mu}^\top \vec{b} \quad \text{s.t.} \quad A^\top \vec{\mu} = -c.$$

6. Homework Process

With whom did you work on this homework? List the names and SIDs of your group members.

NOTE: If you didnt work with anyone, you can put none as your answer.

Answer. none